



Escuela Profesional de
Ciencia de la Computación

Data Base II
2022-A

Almacenamiento: Métodos de Acceso

Ana Maria Cuadros Valdivia

Universidad Nacional de San Agustín de Arequipa

Contenido

- Hash Index
 - Extendible Hash Index
 - Lineal Hash Index

Hash Index

- Son mejores para realizar consultas y selecciones de igualdad.
 - No soporta búsquedas eficientes de rangos
- Se dividen en:
 - Extensible hashing
 - Hashing estática



Extendible Hashing

Extendible Hashing

- Situación: Bucket lleno
 - Evitar overflow de las páginas
- Adicionar más buckets (ej., incrementar N)?
 - Es una opción, pero se necesita una nueva función hash
- Duplicar el número de bucket
 - N valores son potencia de 2: Cómo hacer “mod N”?
 - Qué sucede cuando la función hash es el doble de N?
- Problemas con duplicar:
 - No querer duplicar el tamaño del archivo
 - No querer mover todos los datos.

Extendible Hashing

- **Idea**: Adicionar un nivel de indirección.
- Usar un directorio de punteros a buckets,
- Duplicar el número de bucket al doblar el directorio.
 - El directorio debe ser más pequeño que el file, por lo que duplicarlo es más barato.
- Dividir sólo el bucket que acaba de desbordarse (overflowed)
 - El truco está en cómo se ajusta la función hash.

Extendible Hashing: Cómo trabaja

- **Función Hash:**

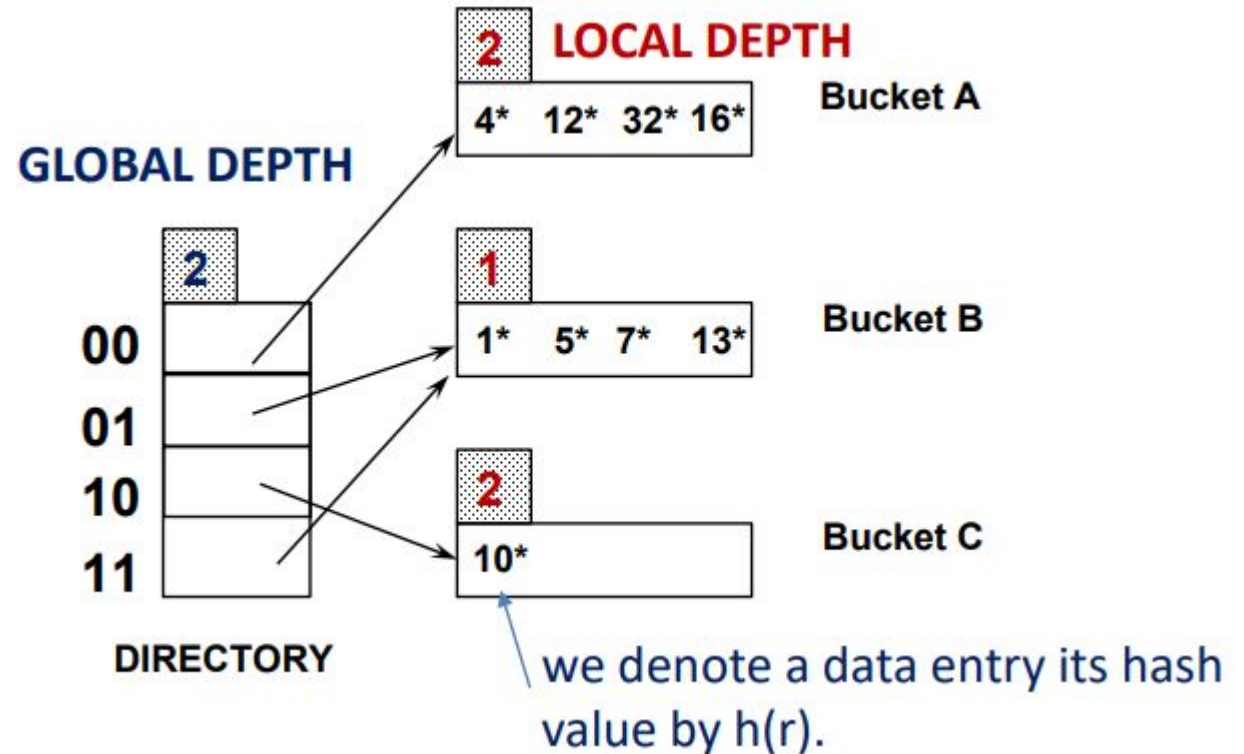
- Define $h(k) = h(k) \bmod 2^D$
- Esencialmente toma los bits más bajos de la clave hash como los valores hash
- El resultado de aplicar una $h(K)$ es tratado como un número binario y los últimos D bits se interpretan como un desplazamiento en el directorio.

- **Directorio:**

- Es un array de punteros (número de páginas) de tamaño 2^D .
- **Global Depth (D)** se conoce como la profundidad global del archivo hash y se mantiene como parte del *header* del archivo.
- Cada puntero apunta a un bucket p_i (no necesariamente únicos).

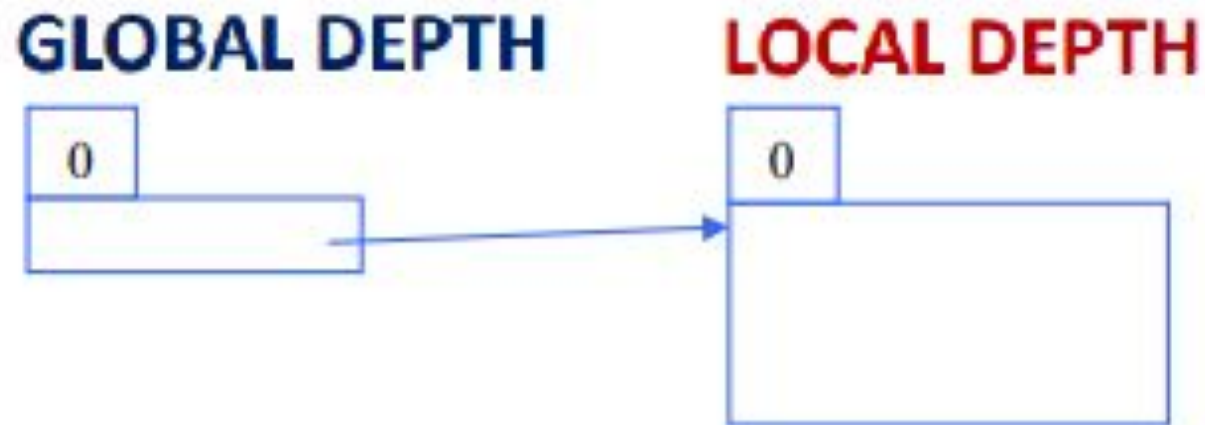
- **Bucket:**

- **Local Depth:** Determina si un bucket está compartido por dos valores hash.



Extendible Hashing: Cómo trabaja

- La estructura inicial del hash es:



Extendible Hashing: Ejemplo 1

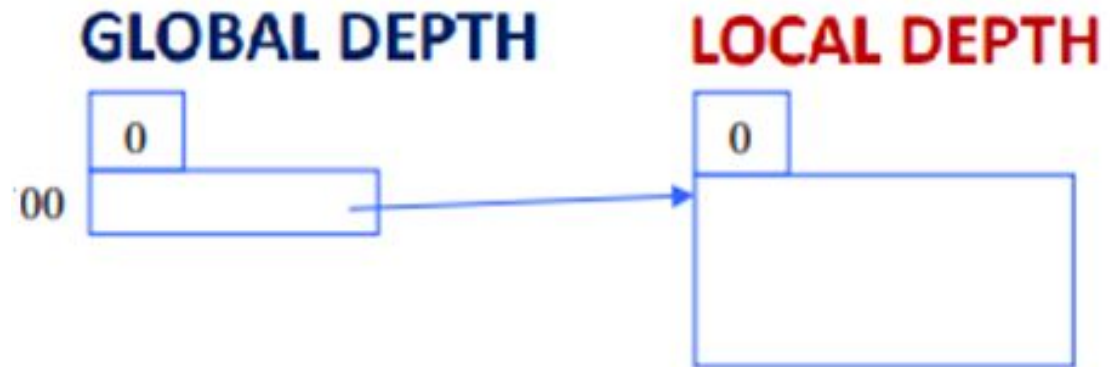
- Suponga que se usa una hash extensible en un archivo de base de datos que contiene registros con los siguientes valores claves de búsqueda:

4, 12, 32, 16, 1, 5, 7, 13

- Construya la estructura hash extensible para este archivo donde cada bucket puede contener 4 registros.

Extendible Hashing: Ejemplo 1

- La estructura inicial es



Extendible Hashing: Ejemplo 1

- Insertar los registros con claves:

4 -> 0010**0**

12 -> 0110**0**

32 -> 1000**0**

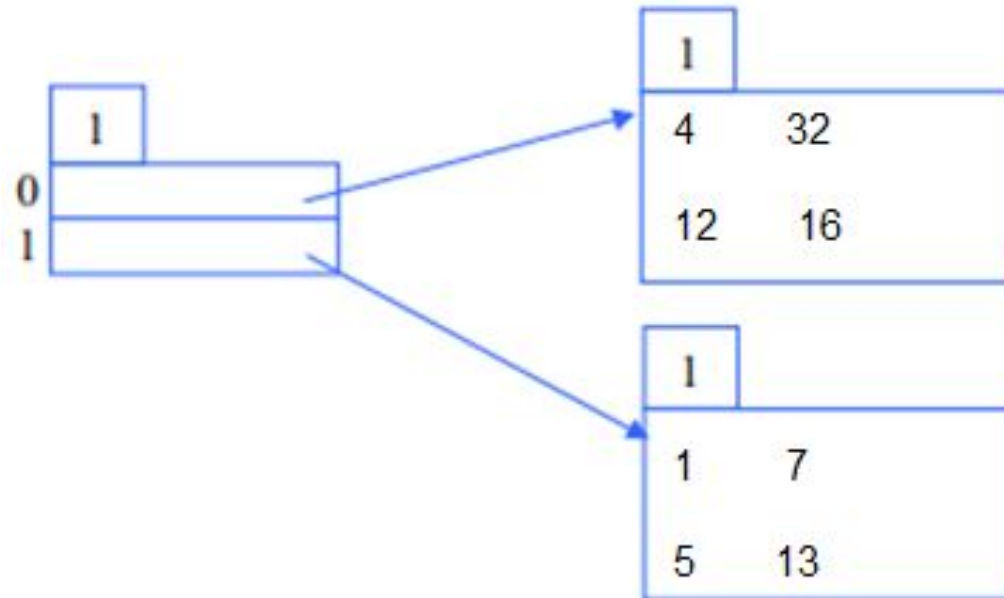
16 -> 0100**0**

1 -> 0000**1**

5 -> 0010**1**

7 -> 0011**1**

13 -> 0100**1**



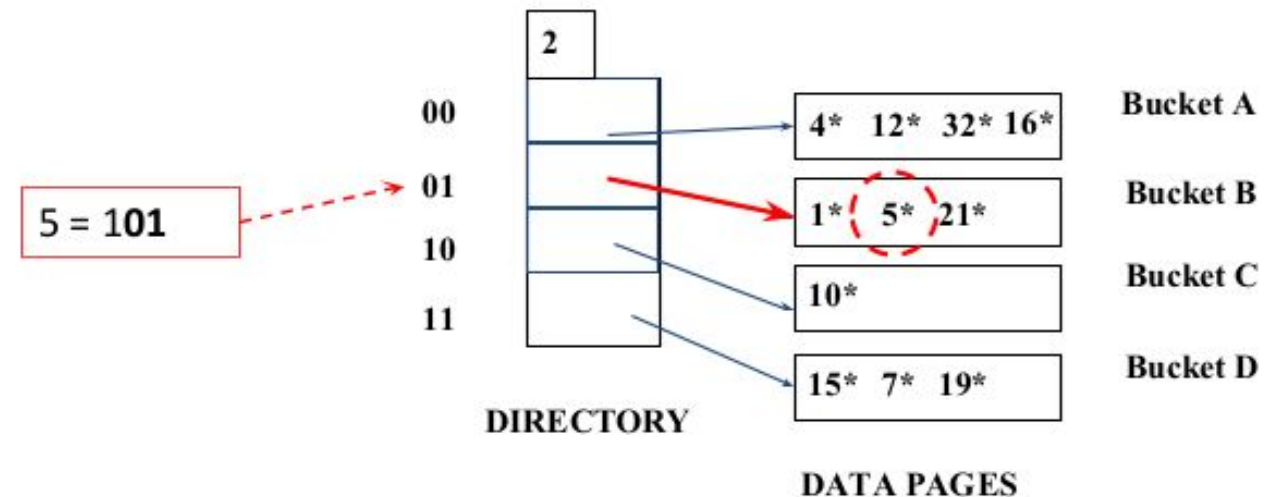
Decimal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110

Extendible Hashing: Ejemplo 1

- **Buscar** por claves:

- Para buscar por elementos, se debe aplicar la función hash a la clave y tomar los últimos d bits de su representación binaria para obtener el número de bucket.

Buscar por 5



Extendible Hashing: Ejemplo 1

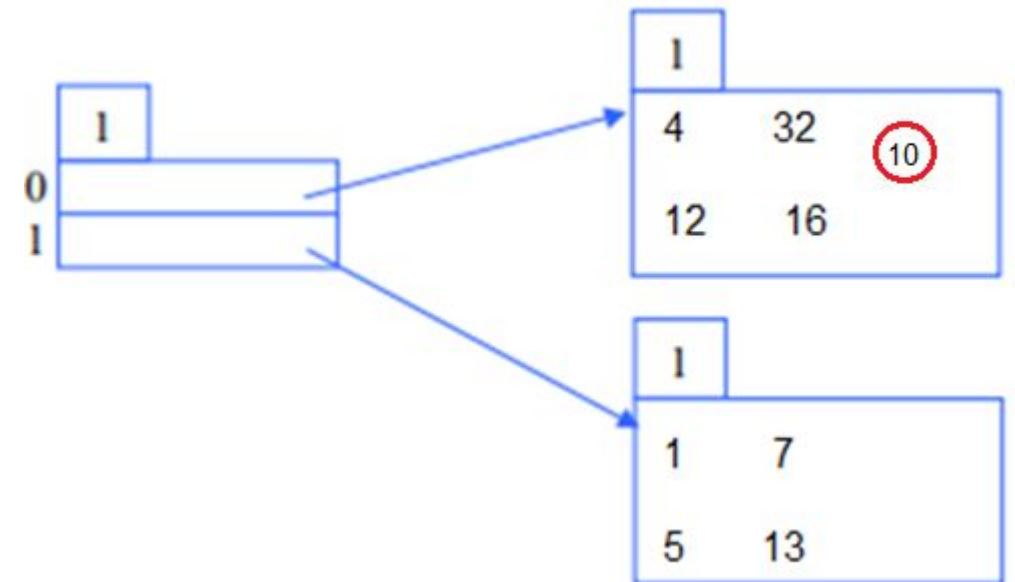
- **Insertar** claves:

- Encontrar el bucket apropiado.
- Dividir el bucket si está lleno y redistribuir su contenido (incluya la nueva entrada que se insertará) entre el bucket lleno y el bucket imagen (el Split bucket).
- Si es necesario, doble el directorio
- Inserte la clave dada.

Extendible Hashing: Ejemplo 1

- Insertar el registro con clave:
 - **10** -> 01010
- El bucket 0 tiene overflowed, entonces se debe dividir el bucket y actualizar (redistribuir) el contenido de ese bucket con 2 bits.

FULL, hence, split and redistribute!



Extendible Hashing: Ejemplo 1

- Actualizar los registros del bucket 0

4 -> 001**00**

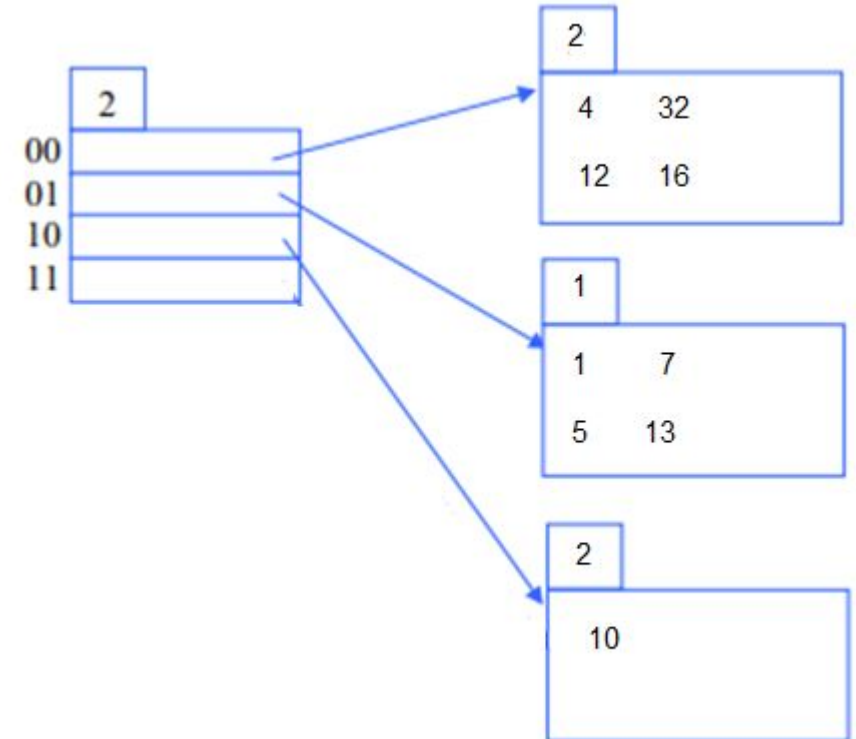
12 -> 011**00**

32 -> 100**00**

10 -> 010**10**

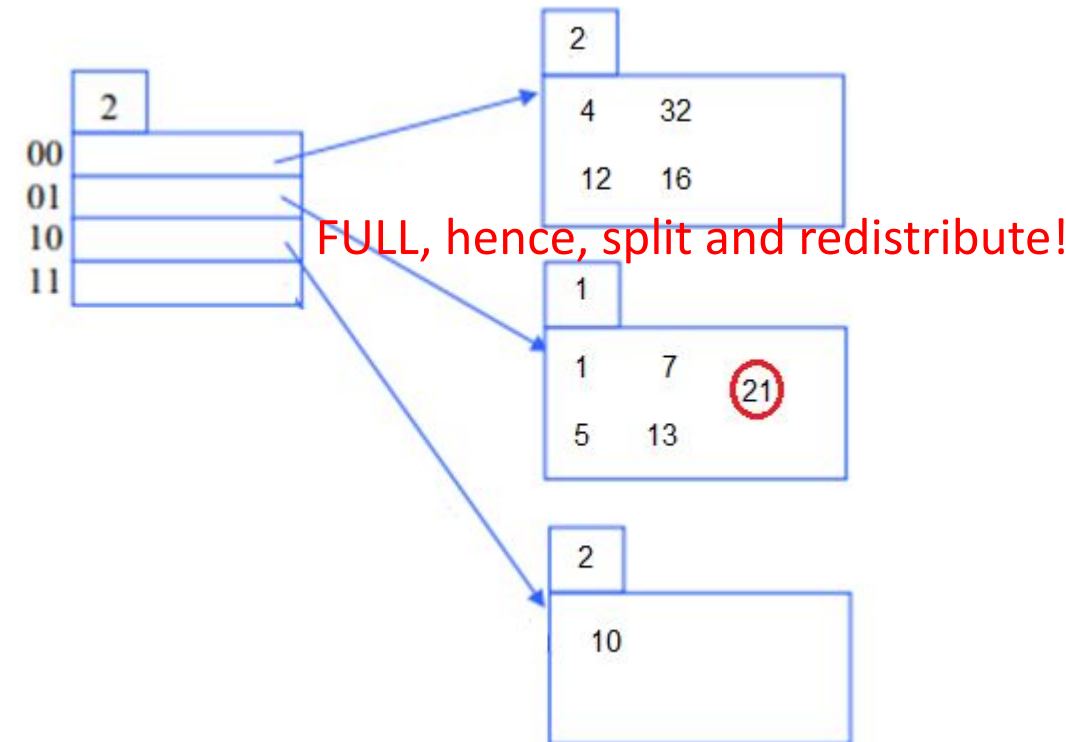
Duplicar el directorio e incrementar el
“Global Depth”

Reasignar los registros a los buckets con
dos bits.



Extendible Hashing: Ejemplo 1

- Insertar el registro con clave:
21 -> 101**01**
- El bucket 01 tiene overflowed,
entonces se debe dividir el bucket y
actualizar el contenido de ese bucket.

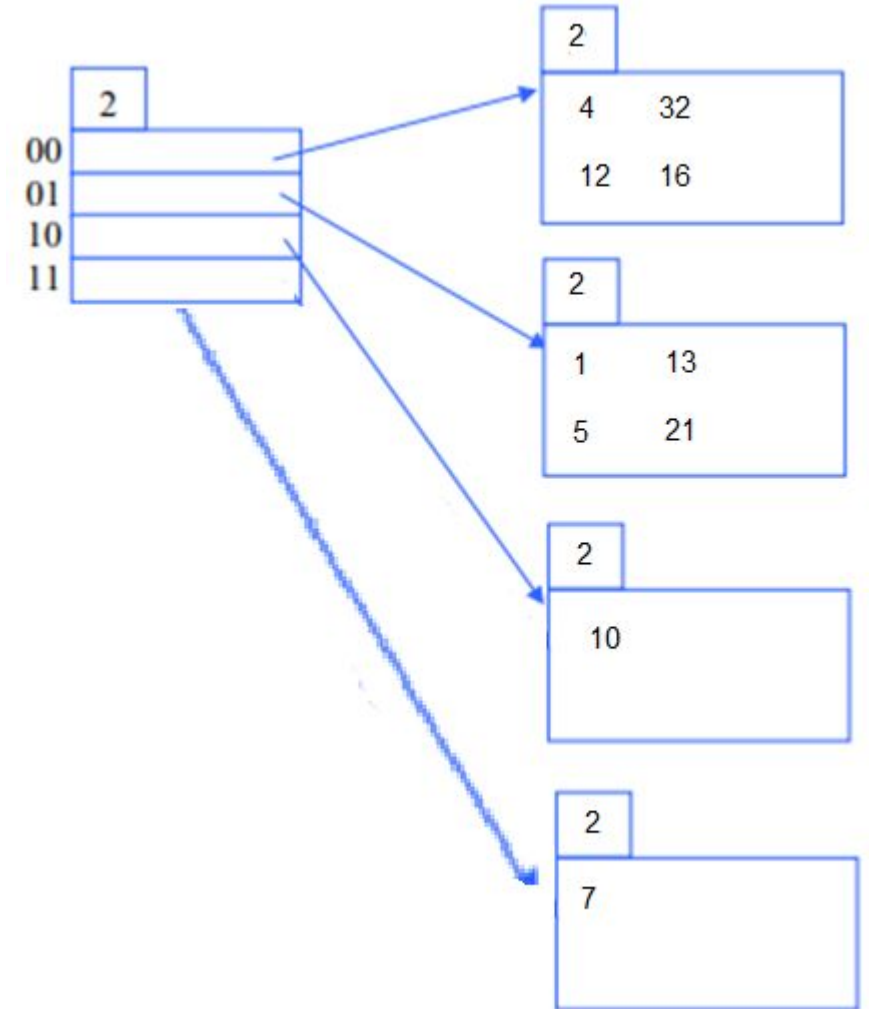


Extendible Hashing: Ejemplo 1

- Actualizar los registros del bucket 01

1 -> 000**01**
5 -> 001**01**
7 -> 001**11**
13 -> 011**01**
21 -> 101**01**

Reasignar los registros a los buckets con dos bits.

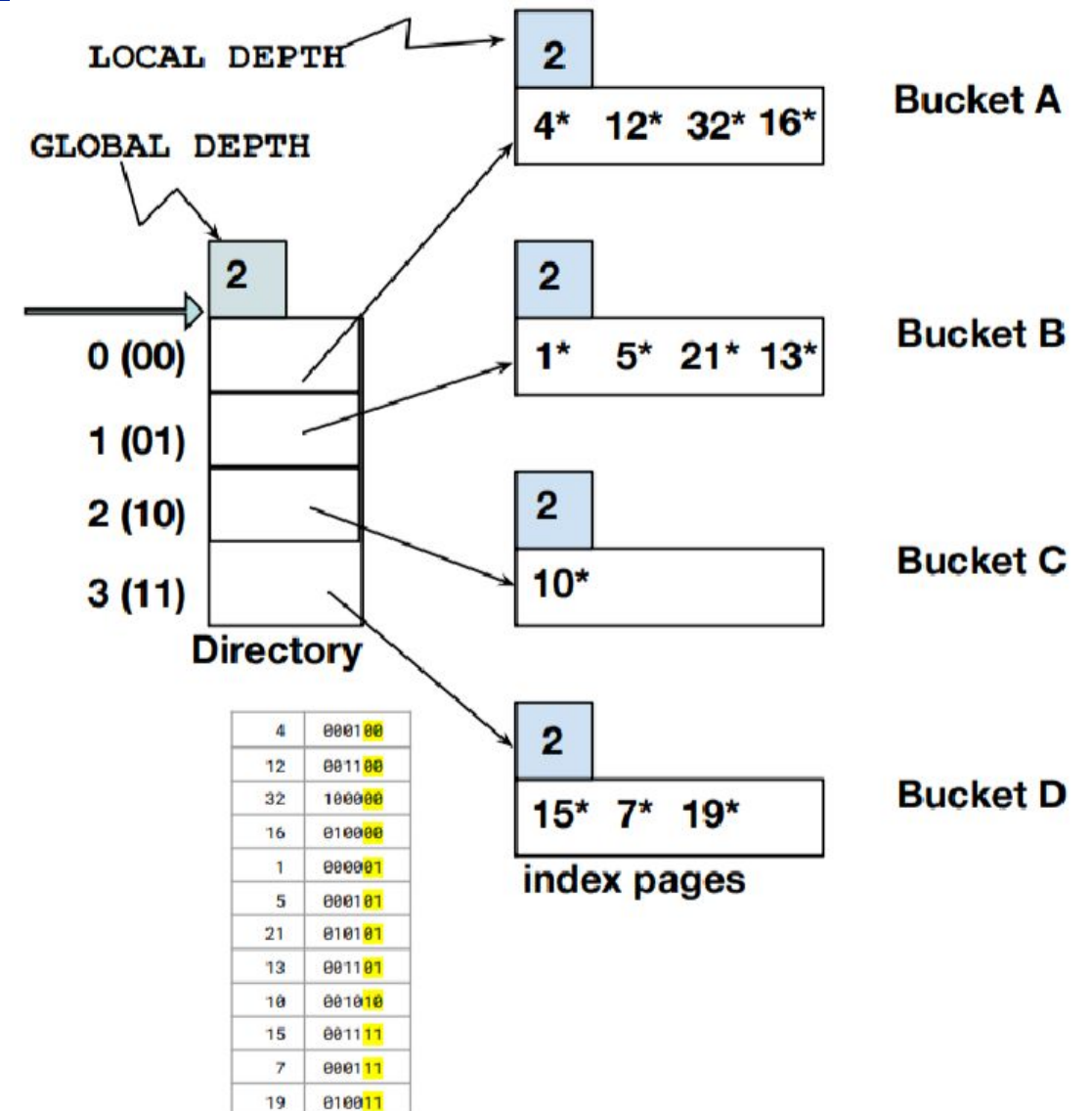


Extendible Hashing: Ejemplo 1

- Insertar el registro con claves:

15 -> 011**11**

19 -> 100**11**



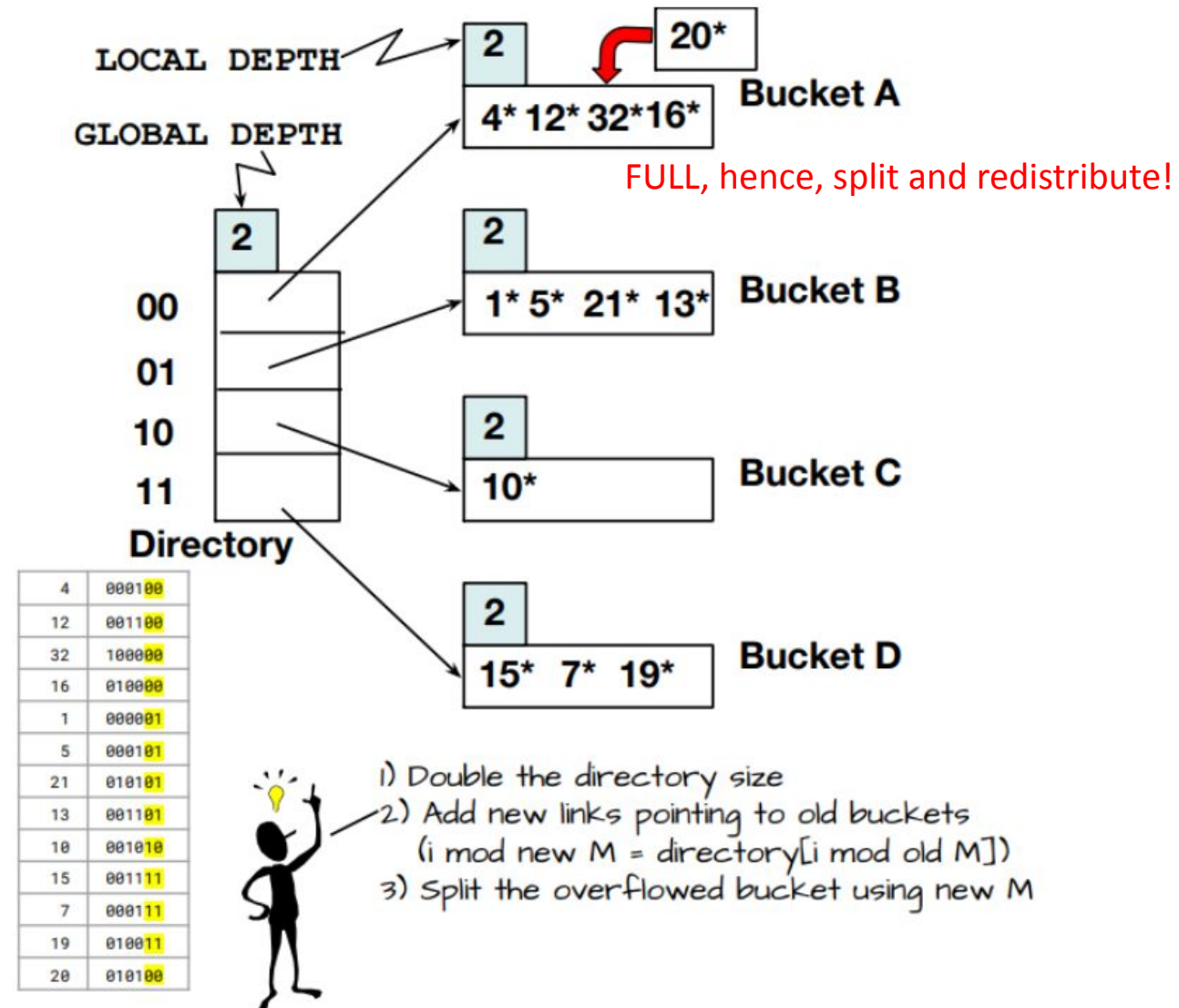
Extendible Hashing: Ejemplo 1

- Insertar el registro con clave:

20 → 10100

Duplicar el directorio e incrementar el “Global Depth”

El bucket 00 tiene overflowed, entonces se debe dividir el bucket y actualizar el contenido de ese bucket con 3 bits.



Extendible Hashing: Ejemplo 1

- Actualizar los registros del bucket 00:

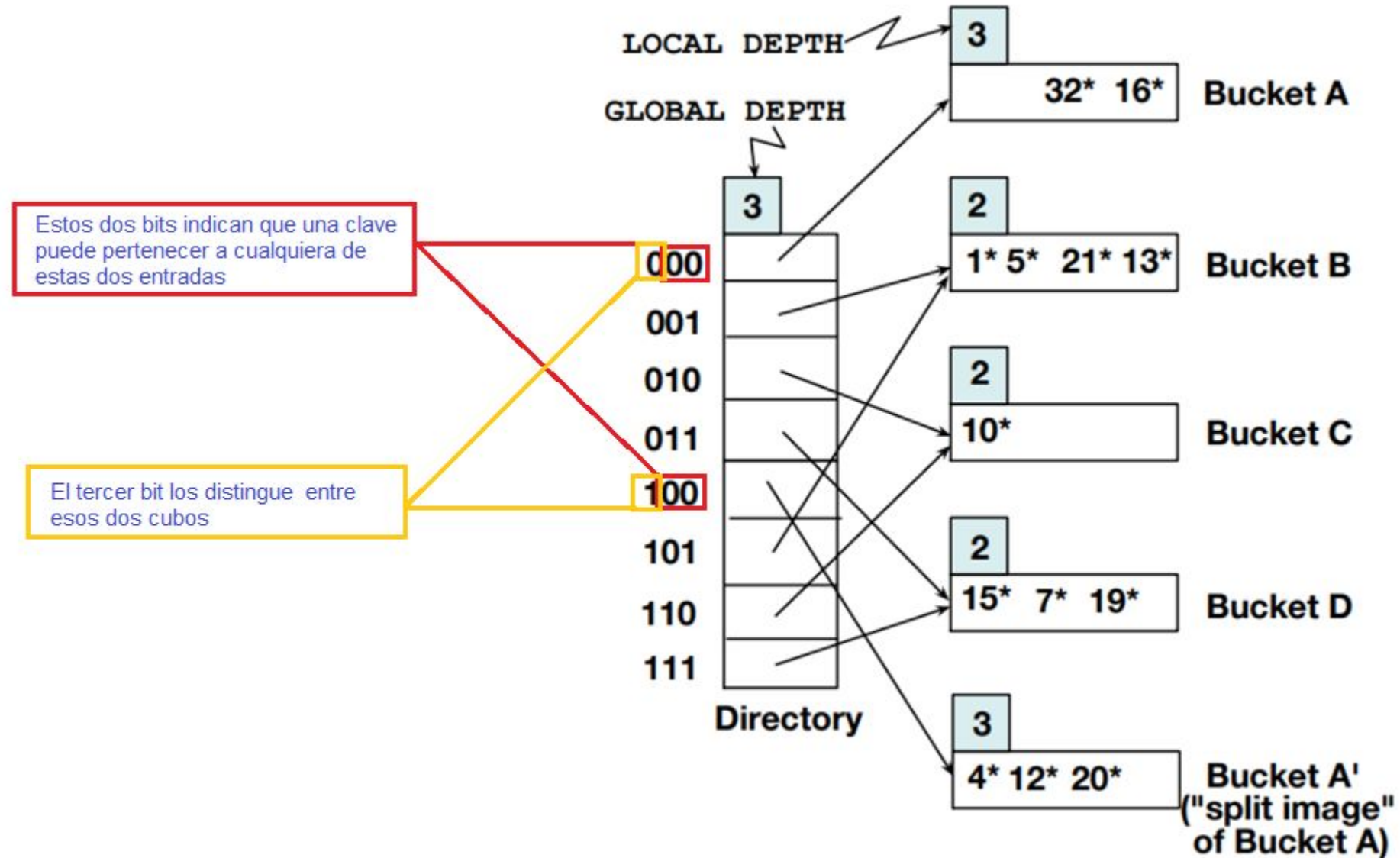
4 -> 00**100**

12 -> 01**100**

32 -> 10**000**

16 -> 01**000**

20 -> 10**100**



Extendible Hashing: Ejemplo 1

- Es necesario siempre duplicar el directorio?
 - Insertar el registro con clave:

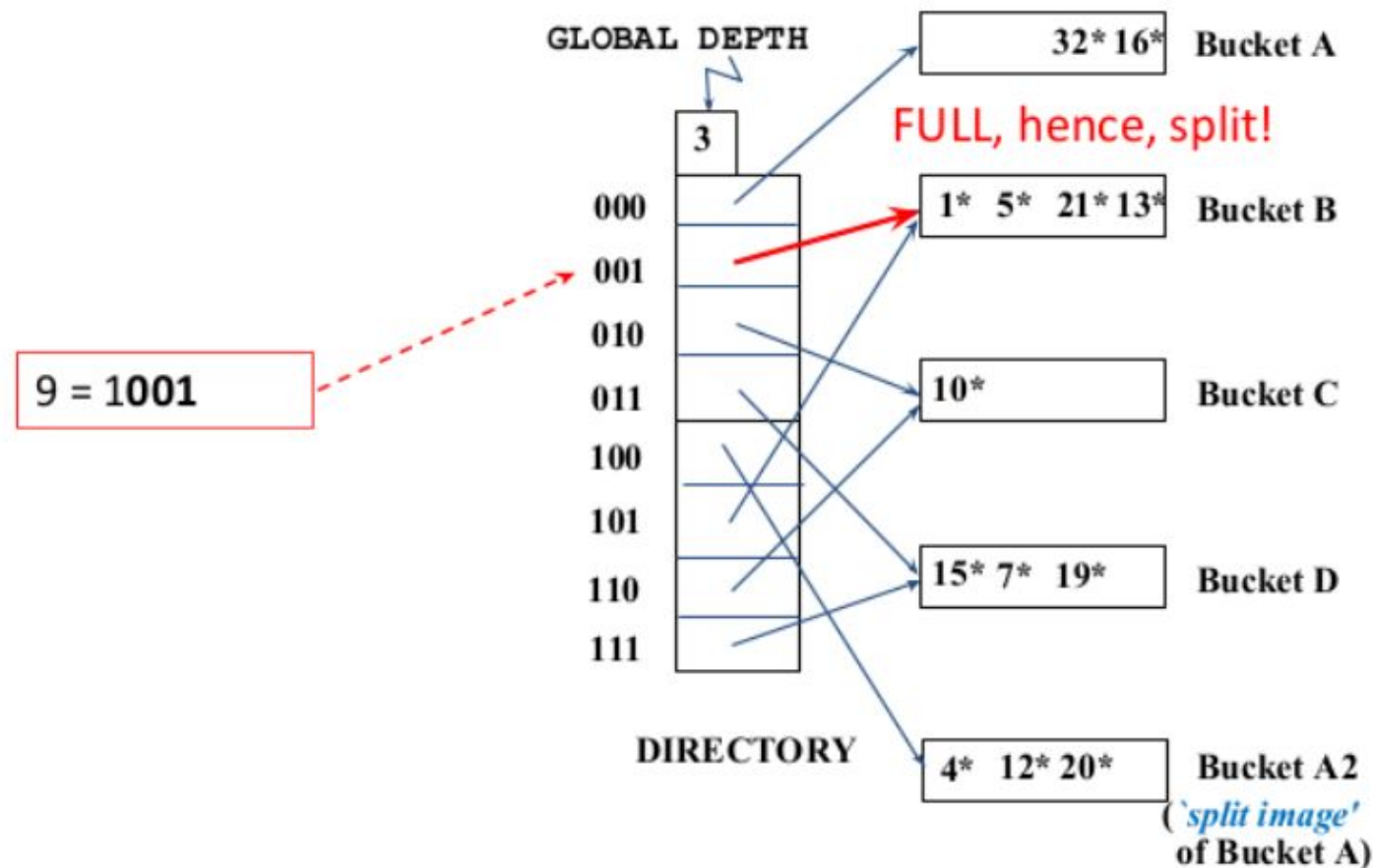
9 -> 1001

1 -> 00001

5 -> 00101

13 -> 01101

21 -> 10101



Extendible Hashing: Ejemplo 1

- Es necesario siempre duplicar el directorio?:
 - Insertar el registro con clave:

9 -> 1001

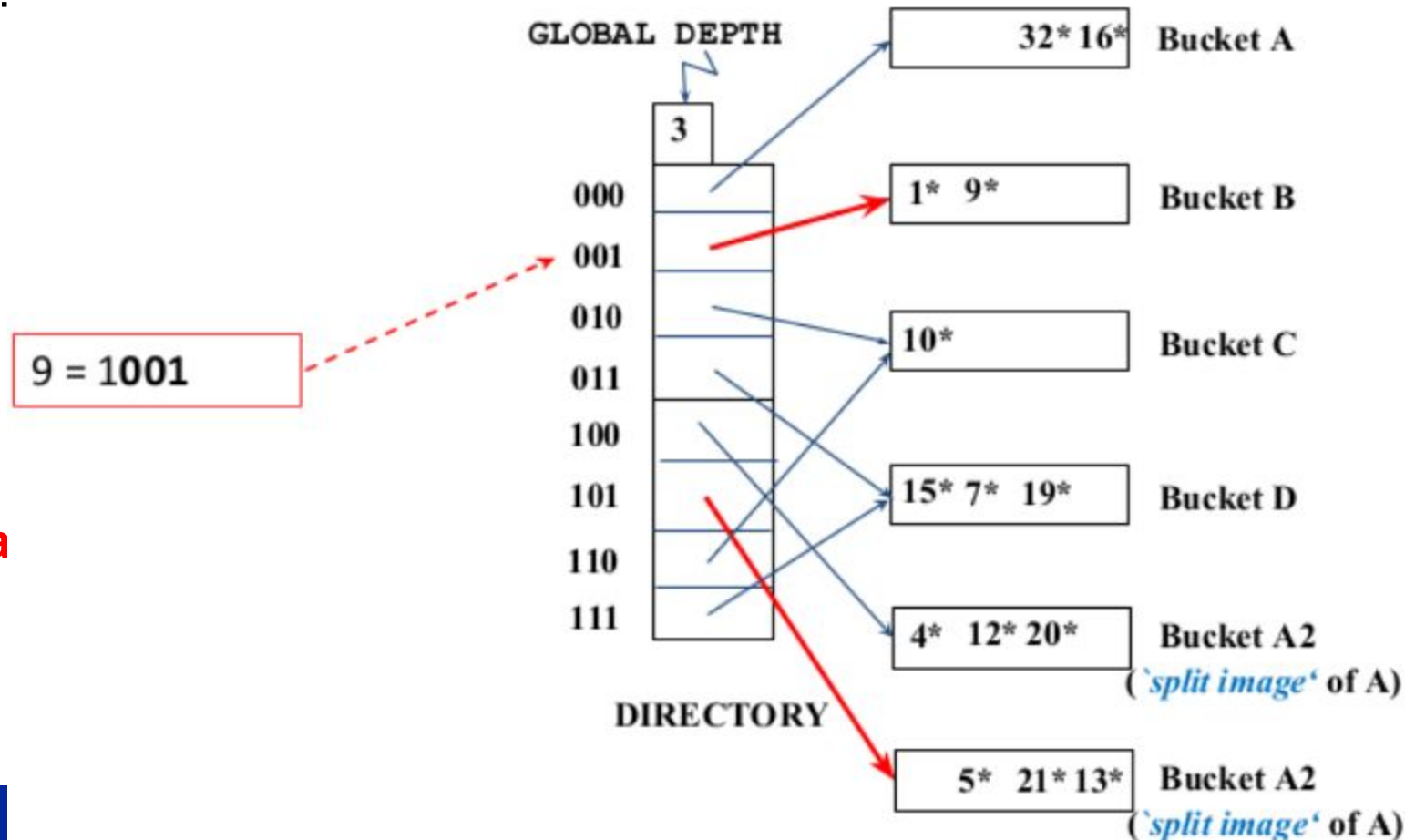
1 -> 0001

5 -> 00101

13 -> 0101

21 -> 10101

No hay necesidad de duplicar el directorio



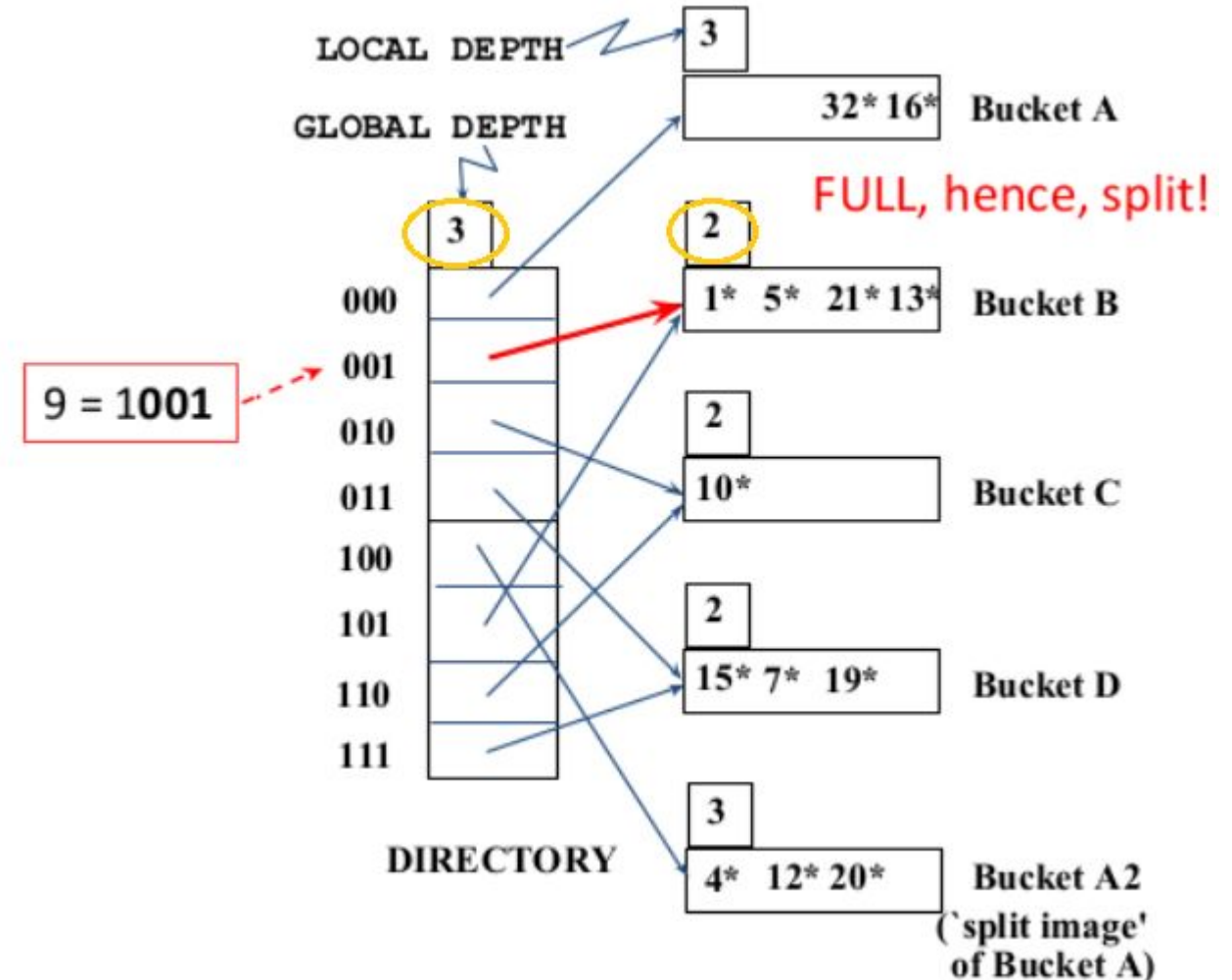
Cuándo se debe duplicar un directorio?

Si el bucket cuya profundidad local “local depth” es igual a la profundidad global “global depth”, el directorio debe duplicarse.

Extendible Hashing

Cuándo se debe duplicar un directorio?

Debido a que LOCAL DEPTH (2) es menor a GLOBAL DEPTH (3), no es necesario duplicar el directorio.



Extendible Hashing

- **Resumen:**

- 4(100), 12(1100) y 20(10100)

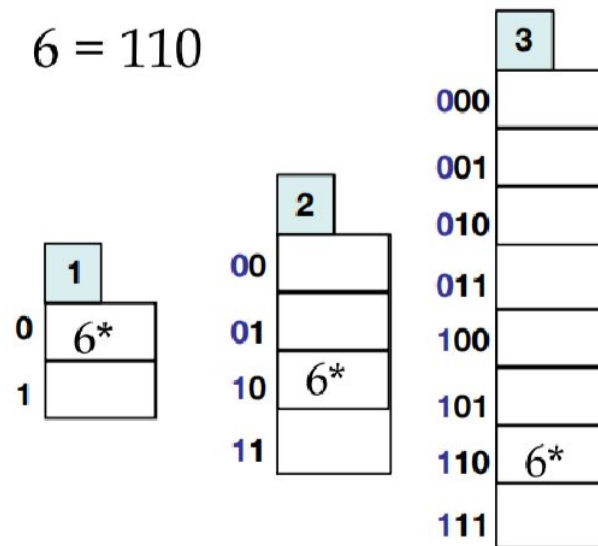
Los últimos 3 bits (100) nos dice que los registros deben de estar en A o A'

- Directorio del profundidad global (***Global Depth***): Máximo # de bits necesarios para saber a qué buckets pertenece un registro.
 - Profundidad local de un bucket (***Local Depth***): Número de bits usados para determinar si un registro pertenece a un bucket.
 - **Cuándo los buckets se dividen y se duplica el directorio?**
 - Antes de insertar, local depth del bucket = global depth.
 - Si la inserción causa que local depth > global depth, el directorio es duplicado, copiándolo y fijando el puntero para dividir la página de la imagen.
 - Usar los bits menos significantes permite una duplicación eficiente.

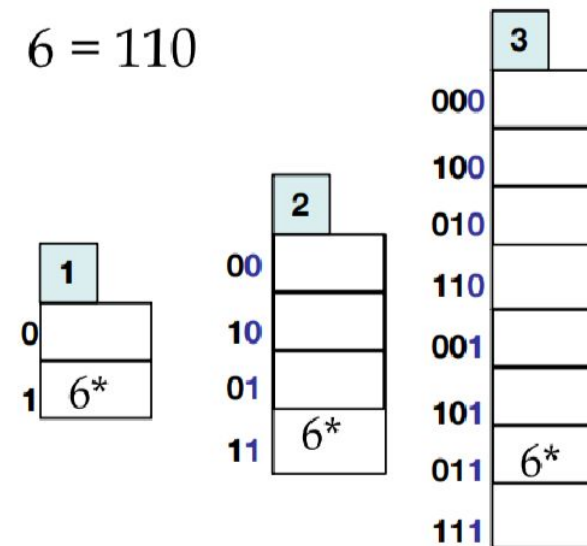
Extendible Hashing

• Porqué usar bits menos significantes en el directorio?

- Permite la duplicación a través de la copia



vs.



Extendible Hashing: Ejemplo 2

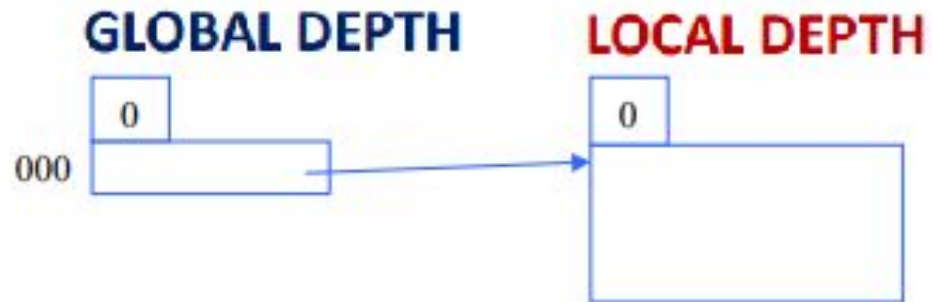
- Suponga que se usa una hash extensible en un archivo de base de datos que contiene registros con los siguientes valores claves de búsqueda:

2, 3, 5, 7, 11, 17, 19, 23, 29, 31

- Construya la estructura hash extensible para este archivo si la función hash es $h(x) = x \bmod 7$ y cada bucket puede contener 3 registros. La clave hash usa tres bits y que los bits comienzan desde el lado izquierdo

Extendible Hashing: Ejemplo 2

- La estructura inicial es

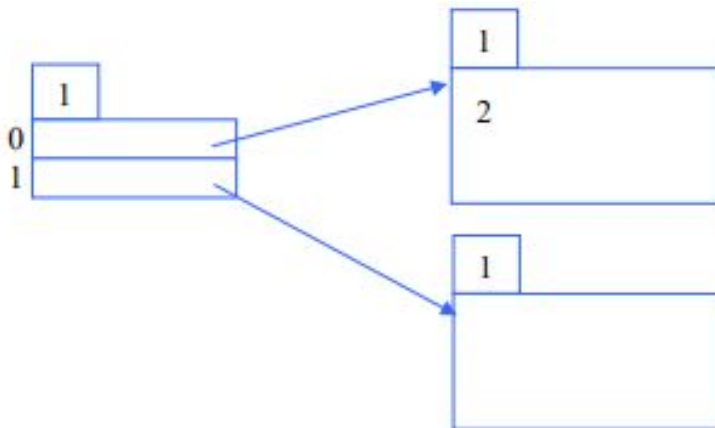


Extendible Hashing: Ejemplo 2

- Insertar el registro 2.

$$2 \pmod{7} = 1$$

1 — 001



Decimal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110

Extendible Hashing: Ejemplo 2

- Insertar los registros con claves:

$3 \rightarrow 3 \bmod 7 = 3 = \mathbf{0}11$, va al bucket 0

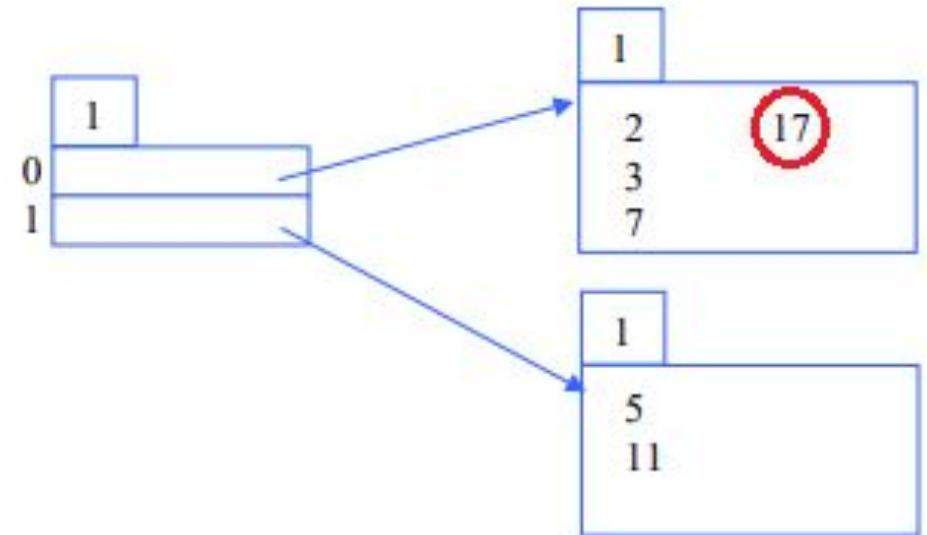
$5 \rightarrow 5 \bmod 7 = 5 = \mathbf{1}01$, va al bucket 1

$7 \rightarrow 7 \bmod 7 = 0 = \mathbf{0}00$, va al bucket 0

$11 \rightarrow 11 \bmod 7 = 4 = \mathbf{1}00$, va al bucket 1

$17 \rightarrow 17 \bmod 7 = 3 = \mathbf{0}11$, va al bucket 0

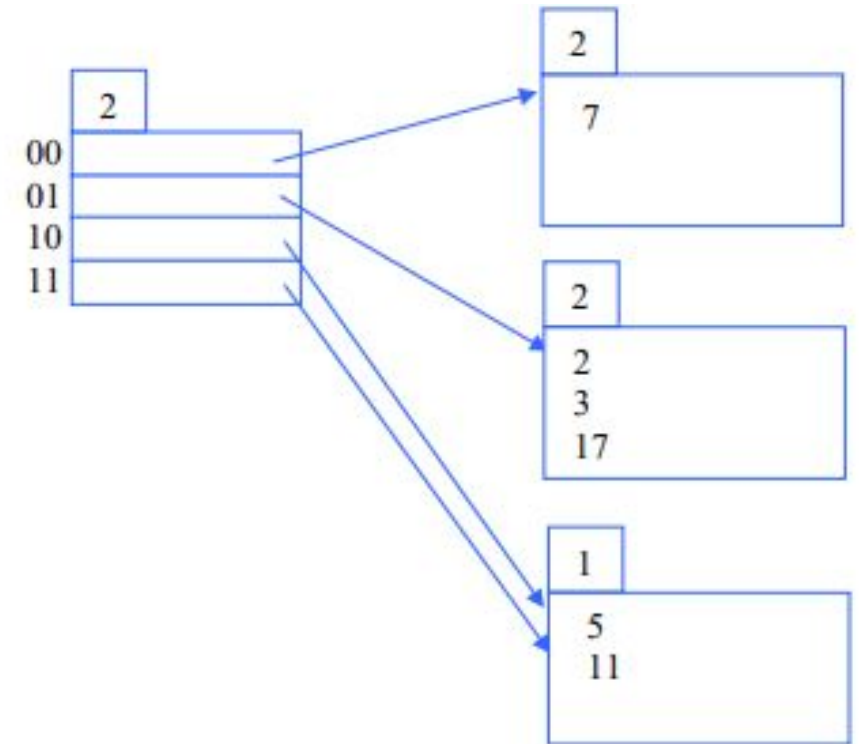
El bucket 0 tiene overflowed, entonces se debe dividir el bucket y actualizar el contenido de ese bucket con 2 bits.



Extendible Hashing: Ejemplo 2

- Actualizar los registros del bucket 0
 - $2 \rightarrow 2 \bmod 7 = 2 = \mathbf{01}0$, va al bucket 01
 - $3 \rightarrow 3 \bmod 7 = 3 = \mathbf{01}1$, va al bucket 01
 - $7 \rightarrow 7 \bmod 7 = 0 = \mathbf{00}0$, va al bucket 00
 - $17 \rightarrow 17 \bmod 7 = 3 = \mathbf{01}1$, va al bucket 01

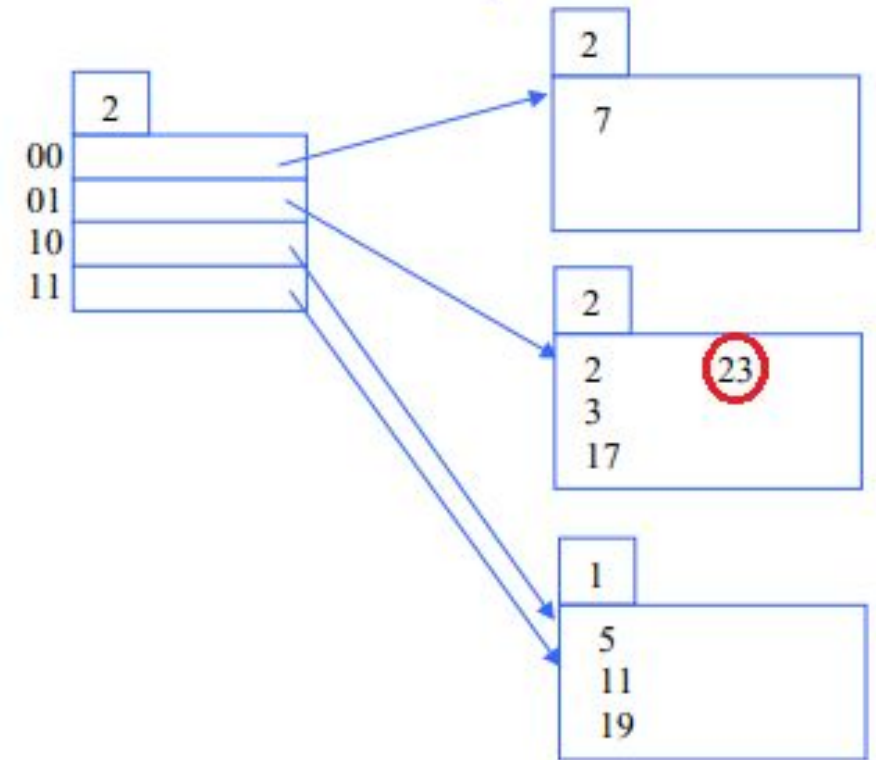
Reasignar los registros a los buckets con dos bits.



Extendible Hashing: Ejemplo 2

- Insertar los registros con claves:
 $19 \rightarrow 19 \bmod 7 = 5 = \mathbf{10}1$, va al bucket 10
 $23 \rightarrow 23 \bmod 7 = 2 = \mathbf{01}0$, va al bucket 01

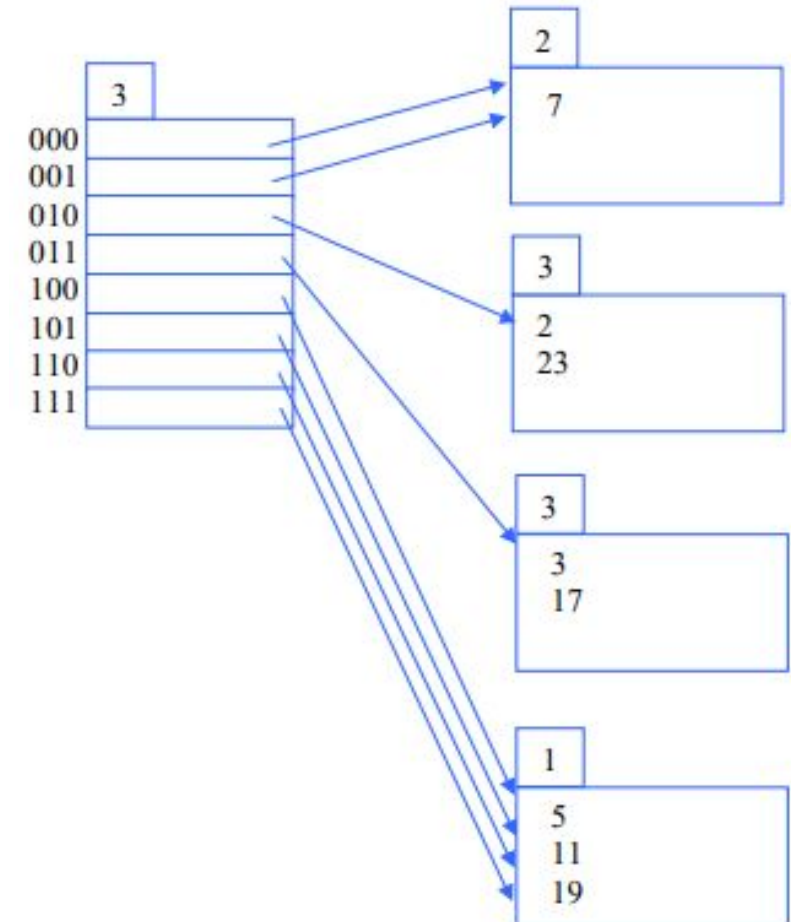
El bucket 01 tiene overflowed, entonces se debe dividir el bucket y actualizar el contenido de ese bucket con 3 bits.



Extendible Hashing: Ejemplo 2

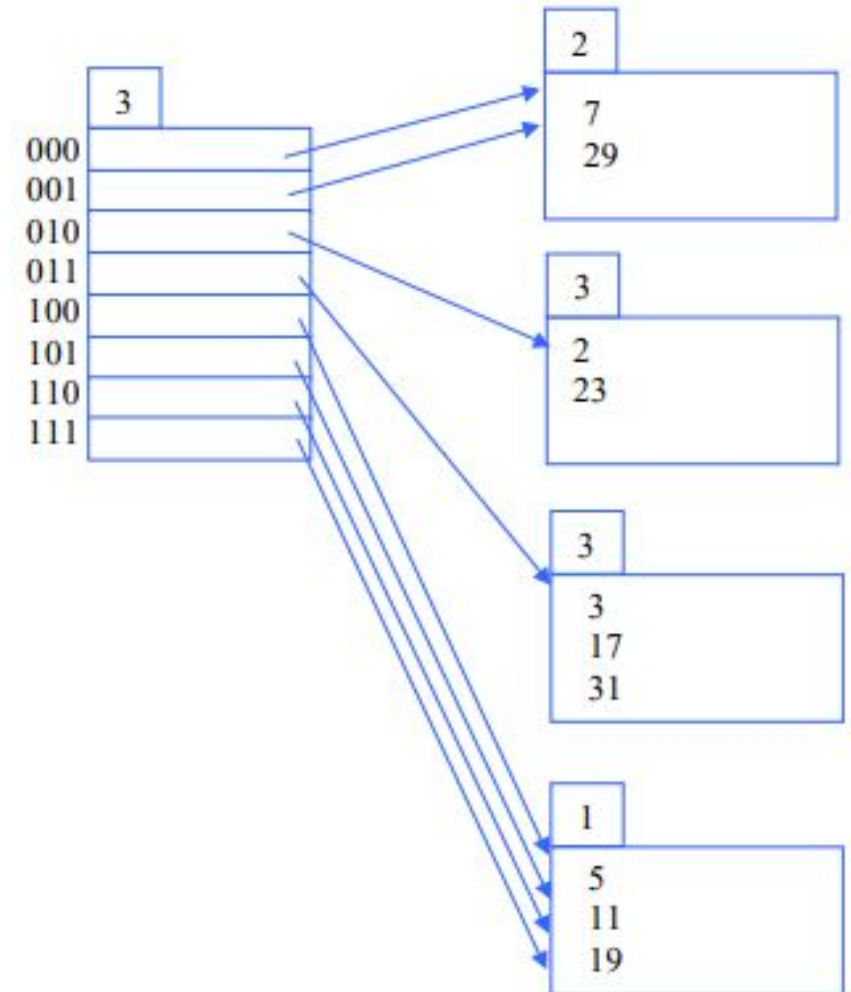
- Actualizar los registros del bucket 01
 - $2 \rightarrow 2 \bmod 7 = 2 = \mathbf{010}$, va al bucket 010
 - $3 \rightarrow 3 \bmod 7 = 3 = \mathbf{011}$, va al bucket 011
 - $17 \rightarrow 17 \bmod 7 = 3 = \mathbf{011}$, va al bucket 011
 - $23 \rightarrow 23 \bmod 7 = 2 = \mathbf{010}$, va al bucket 010

Reasignar los registros a los buckets con tres bits.



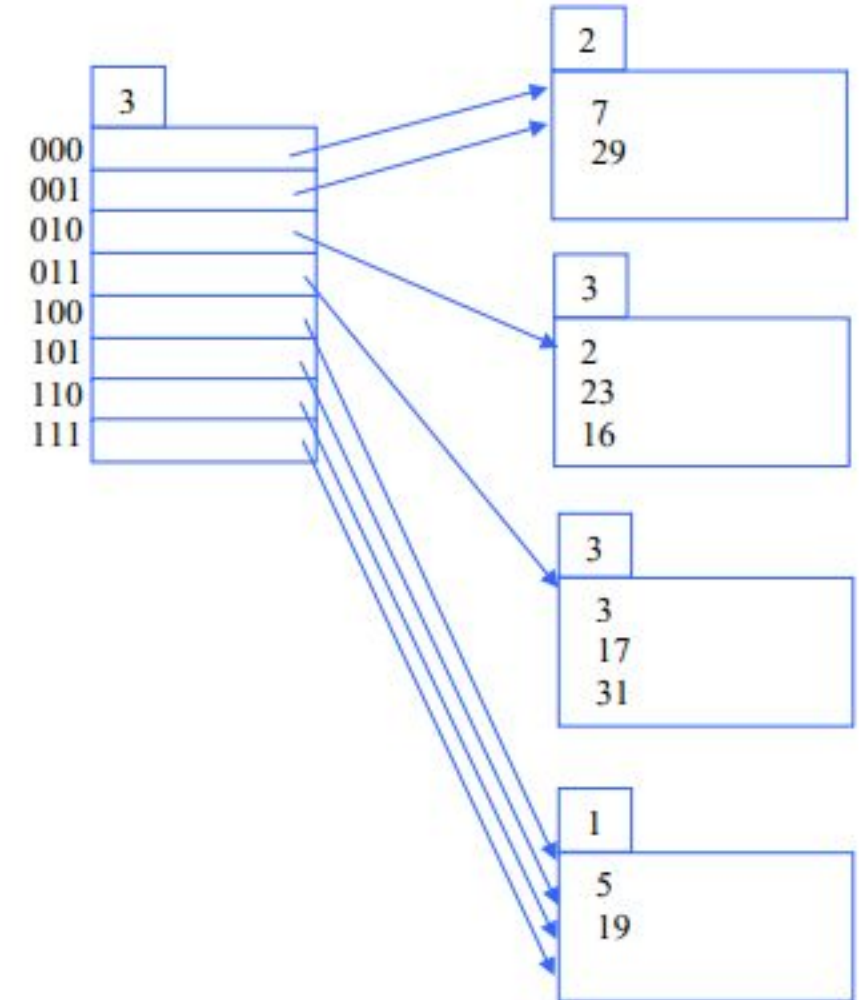
Extendible Hashing: Ejemplo 2

- Insertar los registros con claves:
 $29 \rightarrow 29 \bmod 7 = 1 = \mathbf{001}$, va al bucket 001
 $31 \rightarrow 31 \bmod 7 = 3 = \mathbf{011}$, va al bucket 011
El bucket 01 tiene overflowed, entonces se debe dividir el bucket y actualizar el contenido de ese bucket con 3 bits.



Extendible Hashing: Ejemplo 2

- Insertar el registro con clave :
 $16 \rightarrow 16 \bmod 7 = 2 = \mathbf{010}$, va al bucket 001
- Borrar el registro con clave 11.



Extendible Hashing: Ejemplo 3

- Se puede utilizar la estructura hash para una búsqueda de claves en hay probabilidad de hacer una consulta de un rango de registros?, por ejemplo:

```
select * from relation where key > a and key <=b)
```

Referencias

- Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database system concepts* (Vol. 5). New York: McGraw-Hill.
 - Chapter 14: Index.
- Ramakrishnan, R., Gehrke, J., & Gehrke, J. *Database management systems* (Vol. 3). New York: McGraw-Hill.
 - Chapter FILEORGANIZATIONSANDINDEXES